SOFTWARE REQUIREMENTS SPECIFICATION

Money Manager Application

Version 1.0 • Python Desktop Application

Document Date: March 26, 2026

Project Name	Money Manager App
Document Type	Software Requirements Specification (SRS)
Version	1.0
Language	Python
Architecture	3-Tier + SOLID Principles
Future Plans	Android (Kivy/Flutter), Web (FastAPI + React)
Status	Approved — Ready for Development

. Table of Contents

1	Introduction & Purpose
2	Overall Description
3	Stakeholders & User Classes
4	System Architecture
5	SOLID Design Principles
6	Functional Requirements
7	Non-Functional Requirements
8	Input / Output Specification
9	Use Case Descriptions
10	System Diagrams
11	Database Design
12	Sequence Diagrams
13	Future Enhancements
14	Constraints & Assumptions
15	Revision History

1. Introduction & Purpose

1.1 Purpose

This Software Requirements Specification (SRS) document formally describes the functional and non-functional requirements for the **Money Manager Application** — a Python-based desktop personal finance tool. It serves as the authoritative contract between stakeholders, developers, QA engineers, and future maintainers. All design and implementation decisions must conform to this document.

1.2 Scope

The Money Manager Application enables individuals to track income and expenses, categorize transactions, manage monthly budgets, set personal financial goals, write notes, and view visual reports from an intuitive desktop GUI. The system is built in Python using a strict **3-Tier + SOLID** architecture to ensure maintainability and future extensibility to Android and Web platforms.

1.3 Definitions & Acronyms

SRS	Software Requirements Specification
SOLID	Single Responsibility, Open/Closed, Liskov, Interface Segregation, Dependency Inversion
GUI	Graphical User Interface
CRUD	Create, Read, Update, Delete
DTO	Data Transfer Object
SQLite	Lightweight relational database embedded in Python
Repo	Repository — data access abstraction layer class
KPI	Key Performance Indicator (used in dashboard context)
PyQt5 / Slide6	Python bindings for Qt GUI framework

2. Overall Description

2.1 Product Perspective

The Money Manager Application is a standalone desktop application. In its first version (v1.0), it will run locally on the user's machine with data persisted in a local SQLite database. Future versions will extend this to a cloud-synced mobile (Android) and web application, sharing the same business logic through clean architecture boundaries.

2.2 Product Features

- **Expense & Income Tracking** — Record, edit, and delete financial transactions with categorization.
- **Budget Management** — Define per-category monthly budgets with over-limit alerting.
- **Dashboard & Charts** — Visual summary of financial health with trend lines and pie charts.
- **Notes & Goals System** — Attach notes to entries and track progress toward savings goals.
- **Reports Export** — Generate PDF and Excel reports filtered by date or category.
- **CRUD Operations** — Full create/read/update/delete for all data entities.
- **Multi-currency Support** — User-configurable display currency.

2.3 User Needs

The target user is an individual managing personal or small-household finances who requires a simple, private, offline-first tool. Users need quick transaction entry, clear budget feedback, and actionable financial summaries without reliance on internet connectivity.

3. Stakeholders & User Classes

Stakeholder	Role	Interests
End User	Primary consumer of the app	Ease of use, data privacy, reliability
Developer	Implements features	Clean architecture, testability, clear specs
QA Engineer	Validates correctness	Testable requirements, acceptance criteria
Architect	Owns technical decisions	Scalability, SOLID compliance, future expansion
Product Owner	Defines business requirements	Feature completeness, timeline, UX quality

4. System Architecture

The application follows a strict **3-Tier Architecture**, enforcing clear separation of concerns. Each tier communicates exclusively with the adjacent tier through well-defined interfaces, enabling independent testing and future technology swaps.

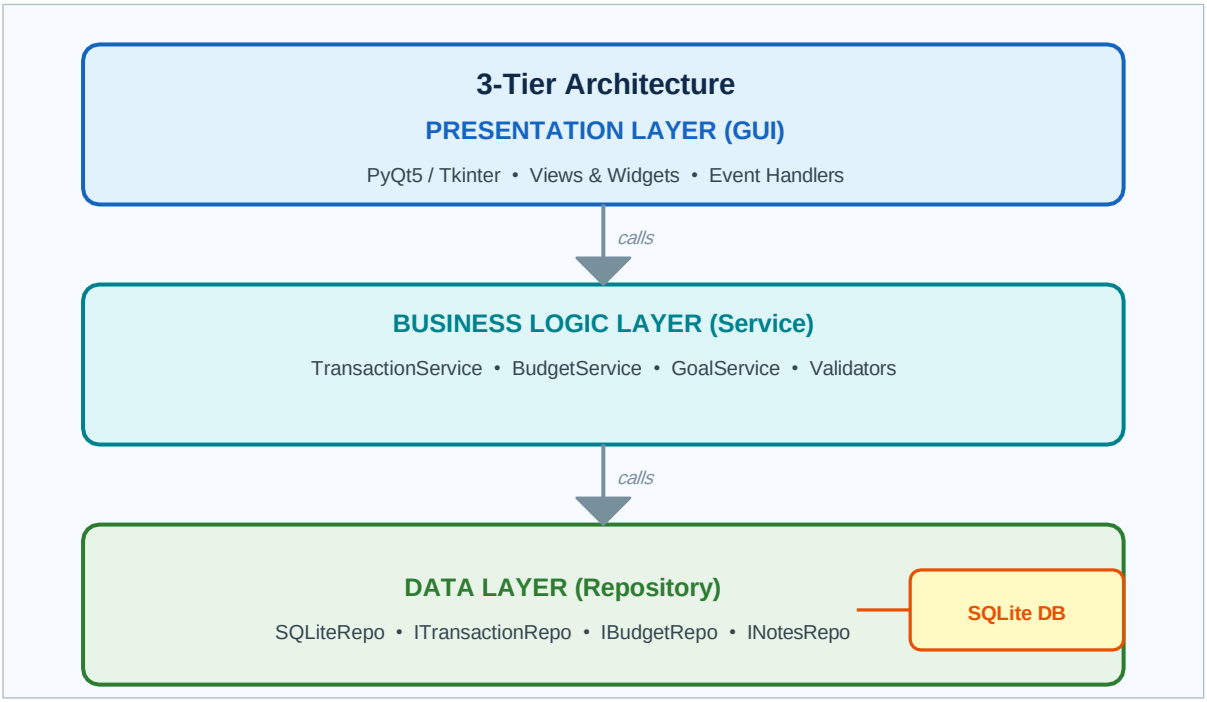

Figure 1 — 3-Tier Architecture Overview

A. Presentation Layer (GUI Layer)

- Renders the graphical user interface using **PyQt5** or **Slide6**.
- Contains no business logic — delegates all processing to the Business Layer.
- Composed of Views, Windows, Dialogs, and Event Handlers.
- Communicates exclusively with Service/Business Layer classes.

B. Business Logic Layer (Service Layer)

- Houses all domain logic: calculations, validations, and transformation rules.
- Examples: **TransactionService**, **BudgetService**,.
- Completely independent of the UI and database implementations.
- Depends only on repository interfaces, never concrete implementations.

C. Data Layer (Repository Layer)

- Manages all data persistence using **SQLite** (upgradeable to API or cloud).
- Defines interfaces: **ITransactionRepo**, **IBudgetRepo**,.
- Concrete repository classes implement these interfaces.
- Can be swapped (e.g., SQLite → REST API) without modifying any business logic.

5. SOLID Design Principles

All components of the Money Manager Application are designed in strict adherence to the **SOLID** object-oriented design principles to maximize maintainability and extensibility.

S — Single Responsibility	Each class has exactly one reason to change. **TransactionService** handles transaction business rules only; **BudgetRepo** handles budget persistence only.
O — Open / Closed	Classes are open for extension but closed for modification. New report types or storage backends are added by creating new classes, not modifying existing ones.
L — Liskov Substitution	Any concrete repository (e.g., **SQLiteTransactionRepo**) can be replaced by another implementation (e.g., **APITransactionRepo**) without breaking the service layer.
I — Interface Segregation	Interfaces are small and focused: **IBudgetRepo**, **ITransactionRepo**. No class is forced to implement methods it does not use.
D — Dependency Inversion	High-level modules (services) depend on abstractions (interfaces), not concrete classes. Dependencies are injected at runtime, enabling easy mocking in unit tests.

6. Functional Requirements

FR-01: Transaction Management

ID	Requirement
FR-01.1	User shall be able to add a new transaction specifying: name, amount, type (income/expense), category, and date.
FR-01.2	User shall be able to view all transactions with filtering by date range, category, and type.
FR-01.3	User shall be able to edit any field of an existing transaction.
FR-01.4	User shall be able to delete a transaction with confirmation prompt.
FR-01.5	System shall auto-categorize transactions based on keyword rules (configurable).

FR-02: Budget Management

ID	Requirement
FR-02.1	User shall define a monthly budget per category with a monetary cap.
FR-02.2	System shall display current spending vs budget limit in real time.
FR-02.3	System shall trigger a visual alert when spending reaches 80% of a budget limit.
FR-02.4	User shall be able to create, update, and delete budget entries.

FR-03: Dashboard & Reports

ID	Requirement
FR-03.1	Dashboard shall display total income, total expenses, and net balance for the current month.
FR-03.2	Dashboard shall render a pie chart of spending by category.
FR-03.3	Dashboard shall render a bar/line chart showing monthly spending trends.
FR-03.4	User shall be able to export financial reports to PDF or Excel format.

FR-04: Notes & Goals

ID	Requirement
FR-04.1	User shall create notes with title, content, date, and optional tags.
FR-04.2	User shall define savings goals with target amount and deadline.
FR-04.3	System shall calculate and display percentage completion of each goal.
FR-04.4	System shall send in-app reminders for approaching goal deadlines.

7. Non-Functional Requirements

ID	Category	Requirement
NFR-01	**Performance**	Application shall launch in under 3 seconds on standard hardware. All GUI interactions shall respond in under 200ms.
NFR-02	**Reliability**	Application shall not lose or corrupt transaction data. All writes shall be atomic via database transactions.
NFR-03	**Usability**	New users shall complete core tasks (add transaction, view report) within 5 minutes without documentation.
NFR-04	**Security**	User password shall be hashed with bcrypt. No plaintext credentials stored on disk.
NFR-05	**Maintainability**	Code coverage by unit tests shall exceed 80%. All public methods shall have docstrings.
NFR-06	**Portability**	Application shall run on Windows 10+, macOS 12+, and Ubuntu 20.04+ without modification.
NFR-07	**Scalability**	Repository layer shall support swapping SQLite for a REST API or cloud database with zero changes to the service layer.
NFR-08	**Data Integrity**	All foreign key constraints shall be enforced at the database level. Cascading deletes shall be configured appropriately.

8. Input / Output Specification

The following diagram illustrates all data inputs consumed by the system and the corresponding outputs produced after processing through the business logic layer.

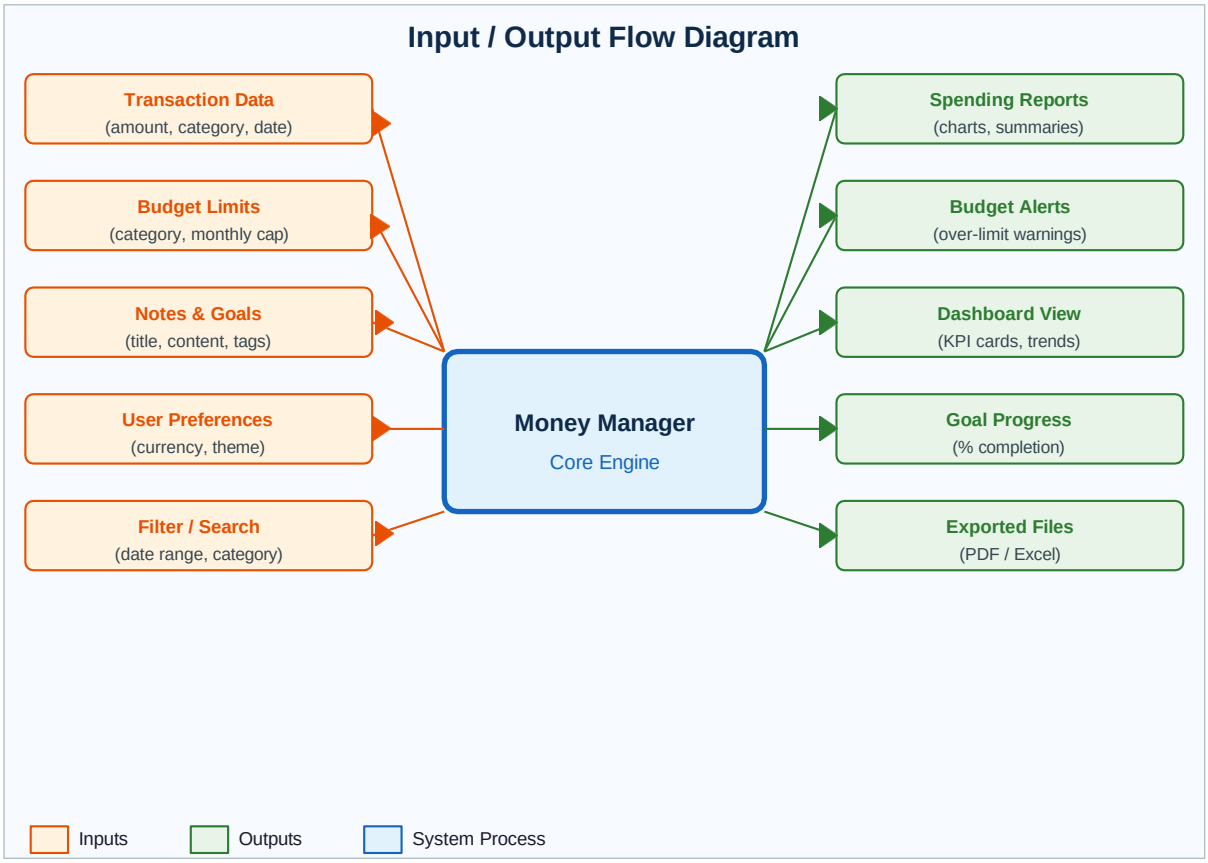

Figure 2 — Input / Output Flow Diagram

Key Data Inputs

Input	Source	Format	Validation Rule
Transaction Data	User Form	Text, Decimal, Date	Amount > 0; Date not in future
Budget Limit	User Form	Decimal, Text	Amount > 0; Category must exist
Note / Goal Entry	User Form	Text, Date	Title required; Deadline > today
Filter / Search Params	GUI	Date Range, String	Start date <= End date
User Preferences	Settings	Enum, String	Currency code must be ISO 4217

9. Use Case Descriptions

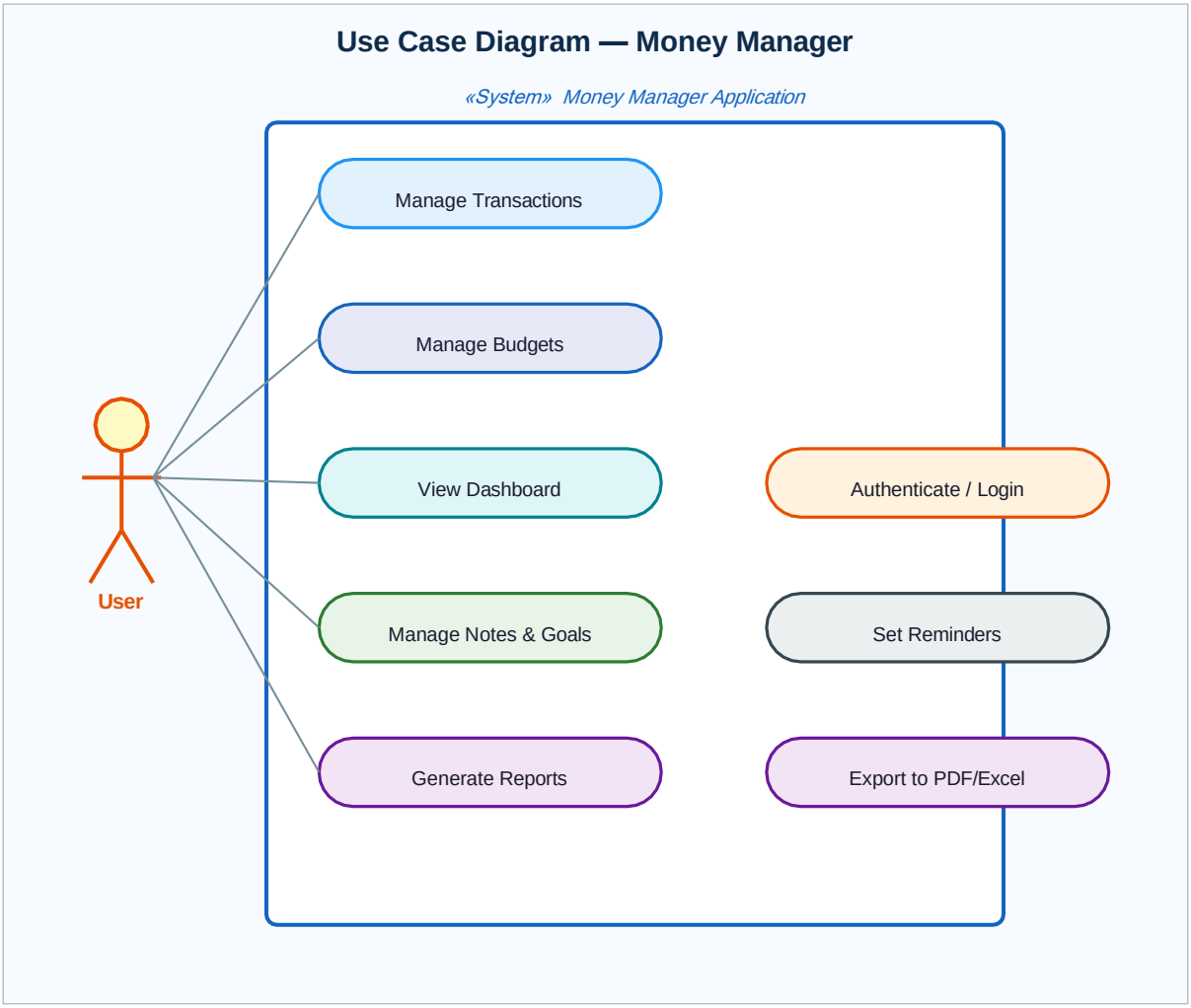

Figure 3 — System Use Case Diagram

UC-01: Add Transaction

Actor	End User
Preconditions	User is logged in; Application is running.
Main Flow	1. User opens Transactions screen. 2. User clicks 'Add New'. 3. System displays transaction form. 4. User fills in name, amount, category, date, type. 5. User submits form. 6. System validates inputs. 7. TransactionService persists record. 8. System refreshes transaction list and dashboard.
Alternate Flow	6a. Validation fails → System highlights error fields; user corrects and resubmits.
Postconditions	New transaction appears in list; dashboard totals updated.

UC-02: Set & Monitor Budget

Actor	End User
Preconditions	User has at least one category defined.
Main Flow	1. User navigates to Budget screen. 2. User selects category and enters monthly limit. 3. System saves budget. 4. As transactions are added, system compares spending vs limit. 5. System displays progress bar per category.
Alternate Flow	5a. Spending >= 80% → System shows orange warning badge. 5b. Spending >= 100% → System shows red over-limit alert.
Postconditions	Budget is saved; spending alerts are active.

10. System Diagrams

10.1 Entity-Relationship Diagram

The following ER diagram shows all database entities and their relationships. All entities are linked to the User entity, enforcing data isolation per user account.

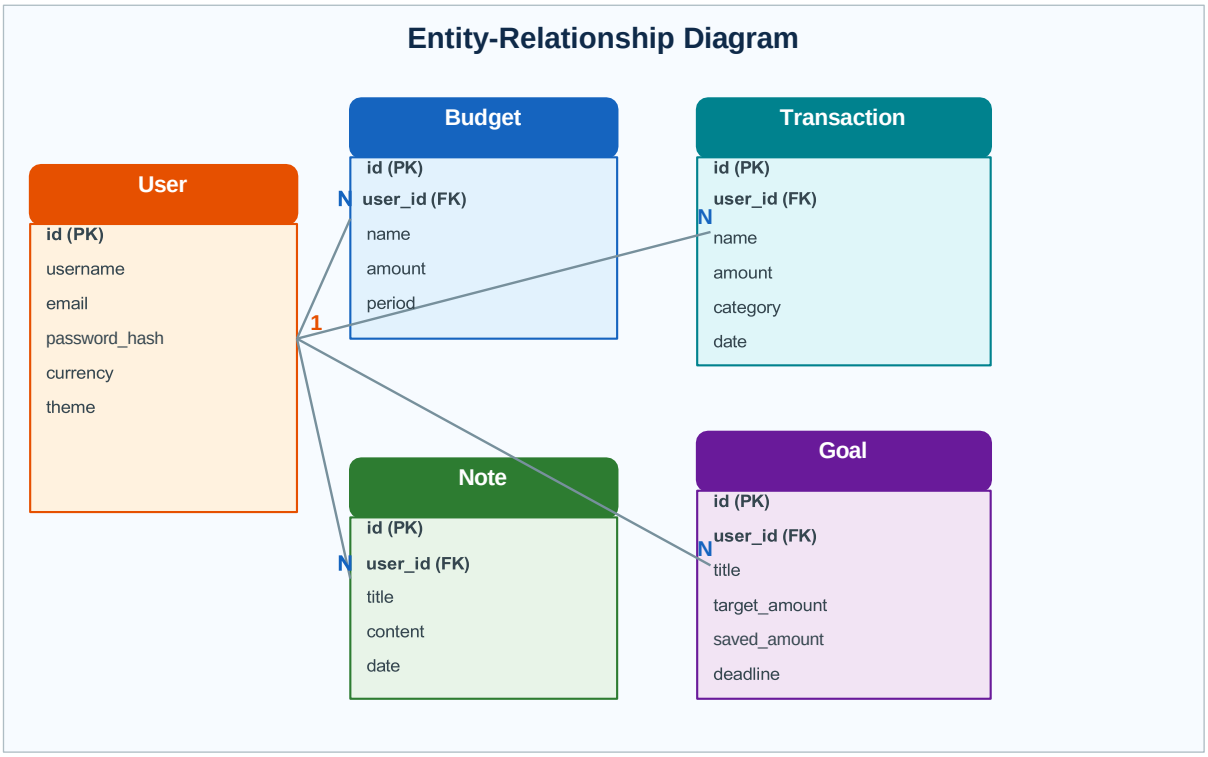

Figure 4 — Entity-Relationship Diagram

11. Database Design

Table: users

Column	Type	Constraints
id	INTEGER	PRIMARY KEY AUTOINCREMENT
username	TEXT	NOT NULL UNIQUE
email	TEXT	NOT NULL UNIQUE
password_hash	TEXT	NOT NULL
currency	TEXT	DEFAULT 'USD'
theme	TEXT	DEFAULT 'light'

Table: transactions

Column	Type	Constraints
id	INTEGER	PRIMARY KEY AUTOINCREMENT
user_id	INTEGER	FOREIGN KEY → users(id) ON DELETE CASCADE
name	TEXT	NOT NULL
amount	REAL	NOT NULL CHECK(amount > 0)
category	TEXT	NOT NULL
date	TEXT	NOT NULL (ISO 8601 format)
type	TEXT	CHECK(type IN ('income','expense'))

Table: budgets

Column	Type	Constraints
id	INTEGER	PRIMARY KEY AUTOINCREMENT
user_id	INTEGER	FOREIGN KEY → users(id) ON DELETE CASCADE
name	TEXT	NOT NULL
amount	REAL	NOT NULL
period	TEXT	e.g. '2024-03' (year-month)

12. Sequence Diagrams

12.1 Add Transaction Flow

The following sequence diagram illustrates the complete message flow when a user adds a new transaction — from GUI event through service validation to database persistence.

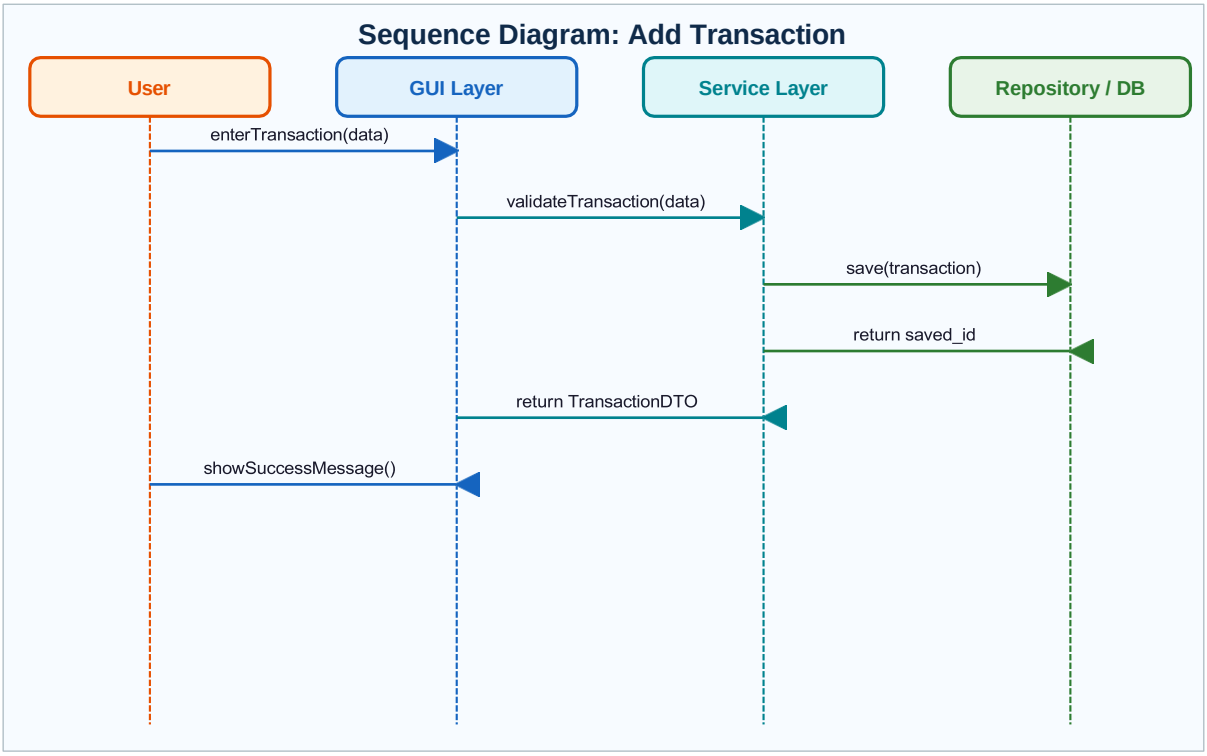

Figure 5 — Sequence Diagram: Add Transaction

13. Future Enhancements

Android Application	Port the Business Logic Layer to a shared Python module. Build native Android UI using **Kivy** or cross-platform mobile with **Flutter** + Python backend.
Web Application	Expose the Business Logic Layer as a RESTful API using **FastAPI**. Build a responsive web frontend using **React** with reusable component library.
Cloud Synchronization	Implement optional cloud sync using **Firebase** or **Supabase**. Replace SQLiteRepo with a CloudRepo implementing the same interfaces — zero service-layer changes.
AI-Powered Insights	Integrate an ML model to classify transactions automatically, detect spending anomalies, and provide personalized saving suggestions based on user history.
PDF / Excel Reports	Full scheduled report generation with charts embedded in PDF output using ReportLab and Excel worksheets with formulas using openpyxl.
Multi-User / Family Mode	Support multiple user profiles per installation with role-based access (admin/viewer) for household budget sharing.

14. Constraints & Assumptions

Constraints

- The initial release (v1.0) supports a single local user; multi-user cloud authentication is deferred to v2.0.
- The GUI must be built using Python-native libraries (PyQt5 or Slide6) — no web technologies in v1.0.
- Data is stored locally in SQLite. Cloud backup is optional and user-initiated.
- Internet connectivity is not required for core functionality.
- The application will be distributed as a self-contained executable (PyInstaller).

Assumptions

- Users have basic familiarity with personal finance concepts (income, expense, budget).
- The development environment uses Python 3.10+ with pip package management.
- SQLite version bundled with Python standard library is sufficient for single-user workload.
- All monetary values are stored as floating-point REAL in SQLite with 2 decimal precision enforced at the application layer.
- Date/time values are stored as ISO 8601 strings (YYYY-MM-DD) in SQLite.

15. Revision History

Version	Date	Author	Description
1.0	2026-03-26	Project Architect	Initial SRS release — all sections complete.
0.2	2024-12-01	Project Architect	Added SOLID principles section and ER diagram.
0.1	2024-11-15	Product Owner	First draft — Introduction and objectives only.